

LAPORAN PROYEK KELAS
MATA KULIAH MANAJEMEN DATA TI2106
SISTEM MANAJEMEN RUANGAN KAMPUS

Pemberian tugas: 29 November 2024



Disusun Oleh :

Kelompok 1

Anggota:

Anisah Ainulhani (13423122)

Najmi Rifdah Hanifah (13423123)

Muhammad Yasin Nurfadllilah (13423136)

PROGRAM STUDI TEKNIK INDUSTRI
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI BANDUNG

2024

KATA PENGANTAR

Puji dan syukur kami panjatkan kepada Tuhan Yang Maha Esa yang telah memberikan karunia-Nya sehingga kami dapat menyelesaikan Laporan Proyek Kelas Mata Kuliah Manajemen Data TI2106 yang berjudul “**SISTEM MANAJEMEN RUANGAN KAMPUS**” ini tepat pada waktunya.

Laporan ini disusun untuk memenuhi tugas mata kuliah Manajemen Data TI2106. Selain itu, laporan ini juga bertujuan untuk memberikan gambaran tentang perancangan sistem yang efektif dalam mengelola pemesanan dan penggunaan ruangan. Penyusunan dokumen ini membantu memecahkan permasalahan manajemen ruangan yang sering dihadapi, seperti pengaturan jadwal, pelacakan penggunaan, dan pengelolaan peran pengguna, dengan menerapkan konsep Entity Relationship Diagram (ERD) yang terstruktur dan sesuai dengan prinsip normalisasi data.

Kami juga mengucapkan terima kasih kepada Ibu Bapak pengampu mata kuliah Manajemen Data yang telah membimbing kami dan memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan kami dalam memecahkan persoalan mengenai manajemen data.

Kami menyadari dalam proses pembuatan tugas dan laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan sebagai pengalaman untuk bekal di masa yang akan datang. Akhir kata, semoga laporan ini dapat bermanfaat dan menambah wawasan, khususnya bagi kami, dan umumnya bagi kita semua.

Cirebon, 9 Desember 2024,

Kelompok 1

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	1
1.4 Manfaat Penelitian	2
BAB II	3
2.1 Porpose	3
2.2 Scope	3
2.2.1 Nama Software	3
2.2.2 Deskripsi Sistem.....	3
2.3 Flowchart Cara Kerja	4
2.4 Product Perspektive	6
2.4.1 Block Diagram.....	6
2.4.2 Use Case	6
2.5 Product Function	7
2.6 ERD (Entity Relationship Diagram)	8
2.7 Program	10
2.8 Query SQL	23
2.8.1 Laporan penggunaan ruangan per hari.....	23
2.8.2 Laporan Peminjaman yang Menunggu Persetujuan	23
2.8.3 Laporan Rekapitulasi Peminjaman Mingguan	24
BAB III	25
3.1 Kesimpulan	25
3.2 Saran	25
3.3 Pembagian Tugas	25
DAFTAR PUSTAKA	26
LAMPIRAN	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era transformasi digital, pengelolaan data yang efektif menjadi kebutuhan utama di berbagai bidang, termasuk pendidikan tinggi. Kampus sebagai institusi pendidikan sering menghadapi tantangan dalam mengelola sumber daya, seperti ruangan, jadwal, dan kebutuhan administratif lainnya. Ketidakefisienan dalam pengelolaan ini dapat mengakibatkan konflik jadwal, pemanfaatan ruangan yang kurang optimal, serta peningkatan beban kerja administratif bagi staf dan mahasiswa.

Untuk menjawab tantangan tersebut, diperlukan sebuah sistem yang mampu mengotomasi pengelolaan data secara efisien dan transparan. Salah satu contoh sistem yang relevan adalah Sistem Manajemen Ruangan Kampus, yang dapat mencatat data ruangan, mengatur jadwal penggunaan, dan memproses pengajuan peminjaman secara terstruktur. Dengan memanfaatkan teknologi seperti Python dan SQL, sistem ini mampu mengintegrasikan berbagai kebutuhan data untuk memberikan solusi yang cepat dan akurat.

Proyek pengembangan sistem ini dirancang untuk memberikan pengalaman langsung kepada mahasiswa dalam memahami konsep manajemen data dan aplikasinya. Melalui simulasi sistem manajemen ruangan, mahasiswa diharapkan mampu mengidentifikasi kebutuhan data, merancang solusi berbasis teknologi, serta menghasilkan laporan yang bermanfaat. Implementasi proyek ini tidak hanya bertujuan untuk meningkatkan kemampuan teknis mahasiswa, tetapi juga memberikan dampak positif pada efisiensi operasional kampus.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem manajemen ruangan kampus yang mampu mencatat, mengelola, dan melaporkan data penggunaan ruangan secara efisien?
2. Bagaimana mengimplementasikan teknologi Python dan SQL untuk mendukung pengelolaan data dalam sistem ini?
3. Bagaimana menghasilkan laporan dalam format CSV yang dapat digunakan untuk analisis lebih lanjut?

1.3 Tujuan

1. Merancang sistem manajemen ruangan kampus yang mampu mencatat data ruangan, jadwal, dan peminjaman secara otomatis.
2. Mengembangkan prototipe sistem berbasis Python dan SQL untuk mendukung operasional pengelolaan data.
3. Menghasilkan laporan yang relevan dan akurat untuk memudahkan analisis dan pengambilan keputusan.

1.4 Manfaat Penelitian

1. Bagi Kampus: Membantu meningkatkan efisiensi dan transparansi dalam pengelolaan ruangan kampus.
2. Bagi Mahasiswa: Memberikan pengalaman langsung dalam merancang dan mengimplementasikan sistem manajemen data berbasis teknologi.
3. Bagi Peneliti Lain: Memberikan referensi dan inspirasi untuk pengembangan sistem serupa di masa mendatang

BAB II

PEMBAHASAN

2.1 Purpose

Sistem manajemen ruangan kampus dirancang untuk mempermudah pengelolaan data ruangan, jadwal penggunaan, dan peminjaman ruangan secara otomatis dan efisien. Sistem ini bertujuan untuk:

1. Meningkatkan efisiensi operasional dalam pengelolaan ruangan kampus.
2. Mengurangi potensi konflik jadwal penggunaan ruangan.
3. Memberikan laporan yang dapat digunakan untuk analisis dan pengambilan keputusan.

2.2 Scope

2.2.1 Nama Software : *Room Management App*

2.2.2 Deskripsi Sistem

Sistem manajemen ruangan kampus adalah aplikasi berbasis Python yang dirancang untuk memfasilitasi pengelolaan data ruangan, penjadwalan, dan peminjaman ruangan secara efisien. Sistem ini menggunakan antarmuka berbasis teks (CLI) dan menyimpan data dalam database SQLite untuk memastikan pengelolaan data yang terorganisir dan mudah diakses.

2.2.2.1 Fitur Utama Perangkat Lunak:

- a. Autentikasi Pengguna
 - Memverifikasi identitas pengguna melalui ID dan kata sandi.
 - Menentukan peran pengguna sebagai operator, mahasiswa, atau dosen, untuk memberikan akses yang sesuai dengan fungsi mereka.
- b. Pengelolaan Ruangan
 - Menambah, mengedit, dan menghapus data ruangan, termasuk ID ruangan, nama ruangan, dan kapasitasnya.
 - Menampilkan daftar ruangan yang tersedia.
- c. Penjadwalan Ruangan
 - Operator dapat membuat, mengedit, atau menghapus jadwal penggunaan ruangan.
 - Jadwal ditampilkan secara kronologis untuk memudahkan pengelolaan.
- d. Pengajuan dan Persetujuan Peminjaman
 - Mahasiswa atau dosen dapat mengajukan peminjaman ruangan dengan memilih ruangan, waktu mulai, dan waktu selesai.
 - Operator meninjau pengajuan dan memberikan keputusan (disetujui/ditolak).
 - Sistem memperbarui status ruangan berdasarkan keputusan.

- e. Pelaporan dan Ekspor Data
 - Operator dapat menghasilkan laporan penggunaan ruangan harian, mingguan, atau bulanan.
 - Data laporan dapat diekspor dalam format CSV untuk dokumentasi atau analisis lebih lanjut.
- f. Pengelolaan Multi-Peran
 - Operator dapat mengelola seluruh aspek sistem (ruangan, jadwal, peminjaman, dan laporan).
 - Mahasiswa dan dosen dapat melihat jadwal ruangan, mengajukan peminjaman, dan melacak status peminjaman mereka.

2.2.2.2 Fungsi Tambahan:

- a. Keamanan Data: Sistem memastikan keamanan data dengan autentikasi pengguna berbasis kata sandi.
- b. Efisiensi Operasional: Sistem dirancang untuk meminimalkan beban kerja manual dan mengotomatiskan banyak proses administratif.
- c. Transparansi: Semua aktivitas tercatat dengan jelas, sehingga memudahkan pelacakan dan audit data.

2.3 Flowchart Cara Kerja

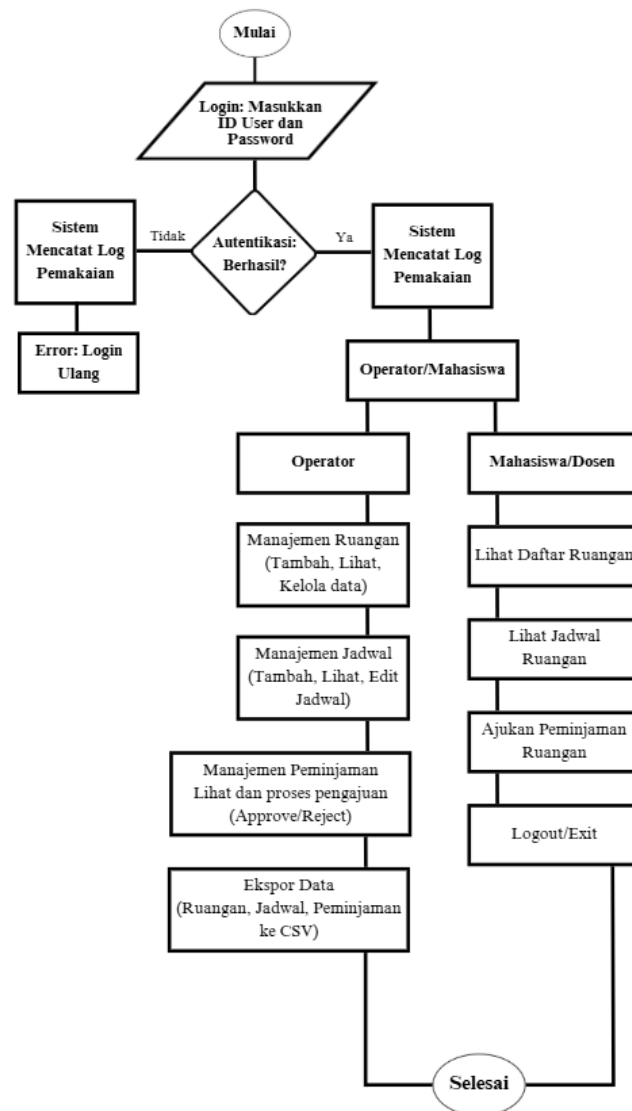
Sistem Manajemen Ruangan Kampus dimulai dengan proses *login*, di mana pengguna memasukkan ID dan password mereka. Sistem kemudian melakukan autentikasi untuk memverifikasi identitas pengguna. Jika autentikasi berhasil, pengguna akan diarahkan ke menu utama sesuai dengan peran mereka, yaitu operator, mahasiswa, atau dosen. Jika autentikasi gagal, sistem akan menampilkan pesan error dan meminta pengguna untuk login kembali.

Untuk operator, sistem menyediakan beberapa fitur utama:

- a. Manajemen Ruangan, yang memungkinkan operator untuk menambah, melihat, dan mengelola data ruangan dengan memasukkan detail seperti ID ruangan, nama, dan kapasitas. Data ruangan yang diperbarui akan disimpan dalam database dan ditampilkan kembali.
- b. Manajemen Jadwal, di mana operator dapat melihat jadwal ruangan yang tersedia dan menambahkan atau mengedit jadwal sesuai kebutuhan.
- c. Manajemen Peminjaman, yang memungkinkan operator untuk menampilkan daftar pengajuan peminjaman, memilih pengajuan tertentu, serta memutuskan untuk menyetujui (*approve*) atau menolak (*reject*) pengajuan tersebut. Status pengajuan akan diperbarui di database.
- d. Ekspor Data, di mana operator dapat mengekspor data ruangan, jadwal, dan peminjaman ke dalam format CSV dan menyimpan file tersebut di lokasi yang diinginkan.
- e. Untuk mahasiswa dan dosen, sistem menyediakan fitur-fitur berikut:
- f. Melihat Daftar Ruangan, yang menampilkan informasi tentang ruangan yang tersedia di kampus.

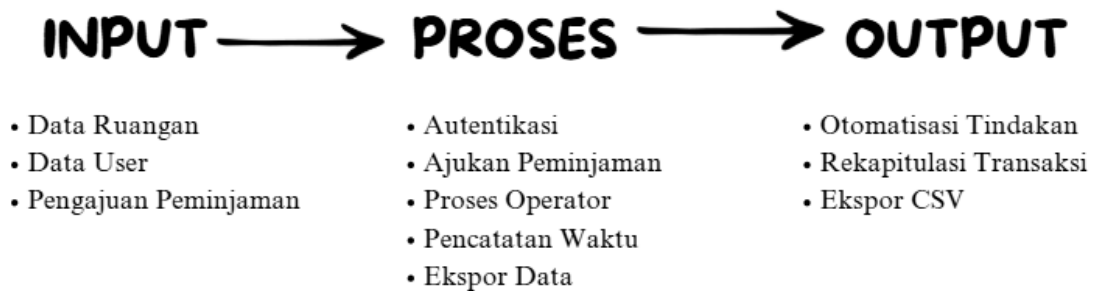
- g. Melihat Jadwal Ruangan, di mana pengguna dapat mengakses jadwal penggunaan ruangan tertentu.
- h. Mengajukan Peminjaman Ruangan, di mana pengguna dapat memilih ruangan dari daftar, menentukan waktu mulai dan selesai, serta memasukkan nama pemesan sebelum mengajukan peminjaman.
- i. Ekspor Jadwal (opsional), di mana pengguna dapat mengekspor jadwal ruangan ke dalam file CSV untuk referensi pribadi.

Sistem ini dirancang agar setiap pengguna dapat menyelesaikan aktivitas mereka dengan mudah dan efisien. Setelah selesai menggunakan sistem, pengguna dapat melakukan logout untuk keluar dari aplikasi. Dengan fitur ini, sistem memberikan kemudahan pengelolaan ruangan dan meningkatkan efisiensi operasional kampus. Berikut merupakan *flowchart* cara kerja sistem yang kami buat:



2.4 Product Perspective

2.4.1 Block Diagram



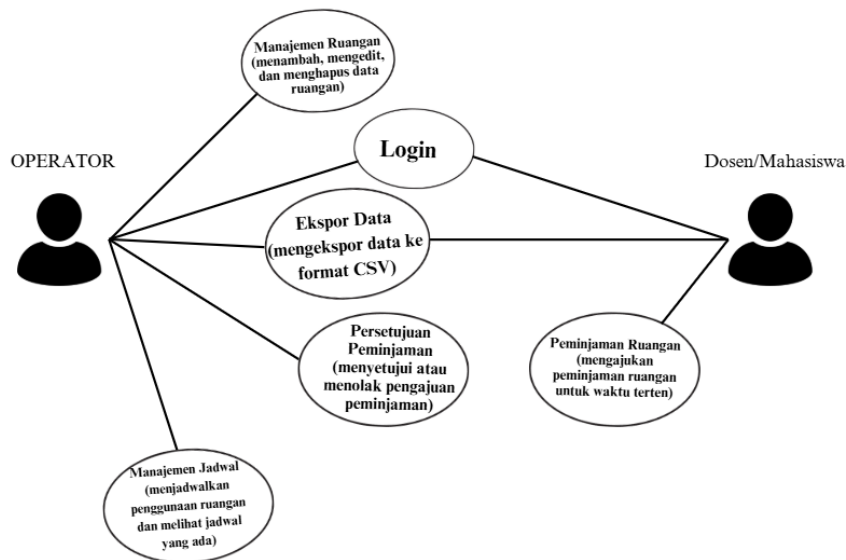
2.4.2 Use Case

a. Author:

- Operator: Bertanggung jawab atas manajemen ruangan, penjadwalan, persetujuan peminjaman, dan ekspor data.
- Mahasiswa/Dosen: Pengguna yang mengajukan peminjaman ruangan dan melihat jadwal penggunaan ruangan.

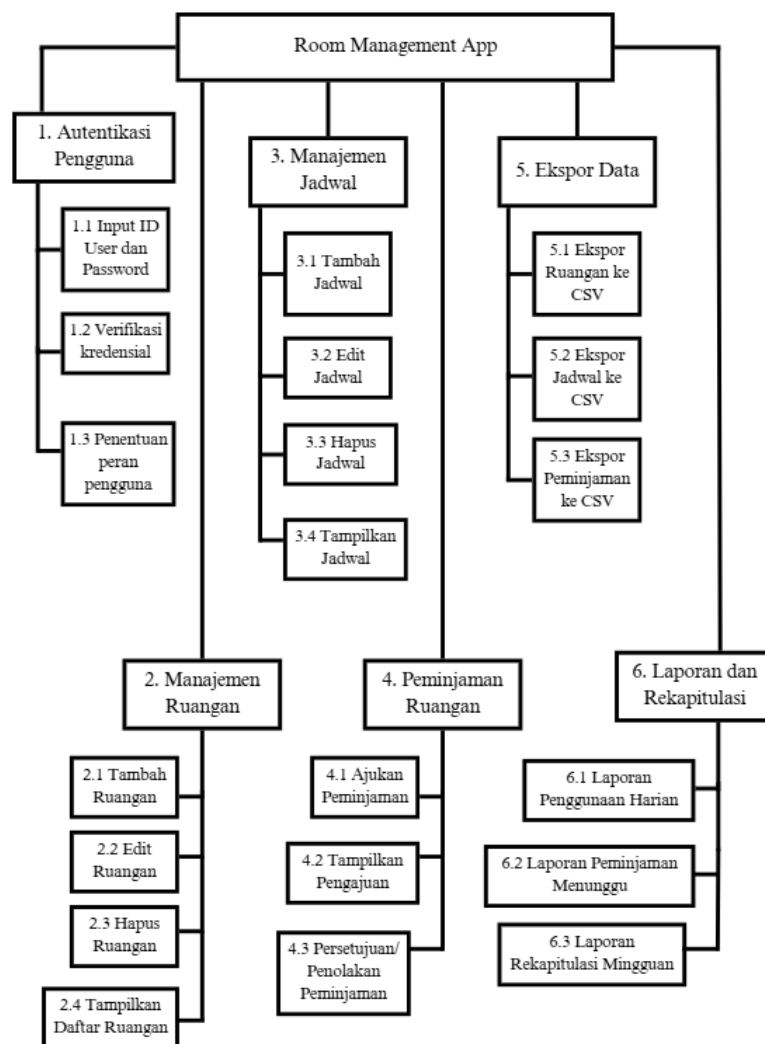
b. Use Case Utama:

- Login: Semua pengguna (Operator, Mahasiswa, Dosen) melakukan autentikasi untuk mengakses sistem.
- Manajemen Ruangan (Operator): Menambah, mengedit, dan menghapus data ruangan.
- Manajemen Jadwal (Operator): Mengelola dan memverifikasi jadwal penggunaan ruangan.
- Peminjaman Ruangan (Mahasiswa/Dosen): Mengajukan peminjaman ruangan.
- Persetujuan Peminjaman (Operator): Menyetujui atau menolak pengajuan peminjaman.
- Ekspor Data: Semua aktor dapat mengekspor data ruangan, jadwal, atau peminjaman dalam format CSV.



2.5 Product Function

Berikut merupakan Dekomposisi Modul dari sistem Sistem Monitoring Penggunaan Ruang di Kampus.



2.6 ERD (*Entity Relationship Diagram*)

a. Entitas dan Atribut

1. User
 - ID_User (Primary Key)
 - Nama
 - Password
 - Role
2. Ruangan
 - ID_Ruangan (Primary Key)
 - Nama_Ruangan
 - Kapasitas
3. Jadwal
 - ID_Jadwal (Primary Key)
 - ID_Ruangan (Foreign Key)
 - Waktu_Mulai
 - Waktu_Selesai
 - Status
 - Pemesan
 - Status_Approval
4. Peminjaman
 - ID_Peminjaman (Primary Key)
 - ID_Ruangan (Foreign Key)
 - Waktu_Mulai
 - Waktu_Selesai
 - Pemesan
 - Status

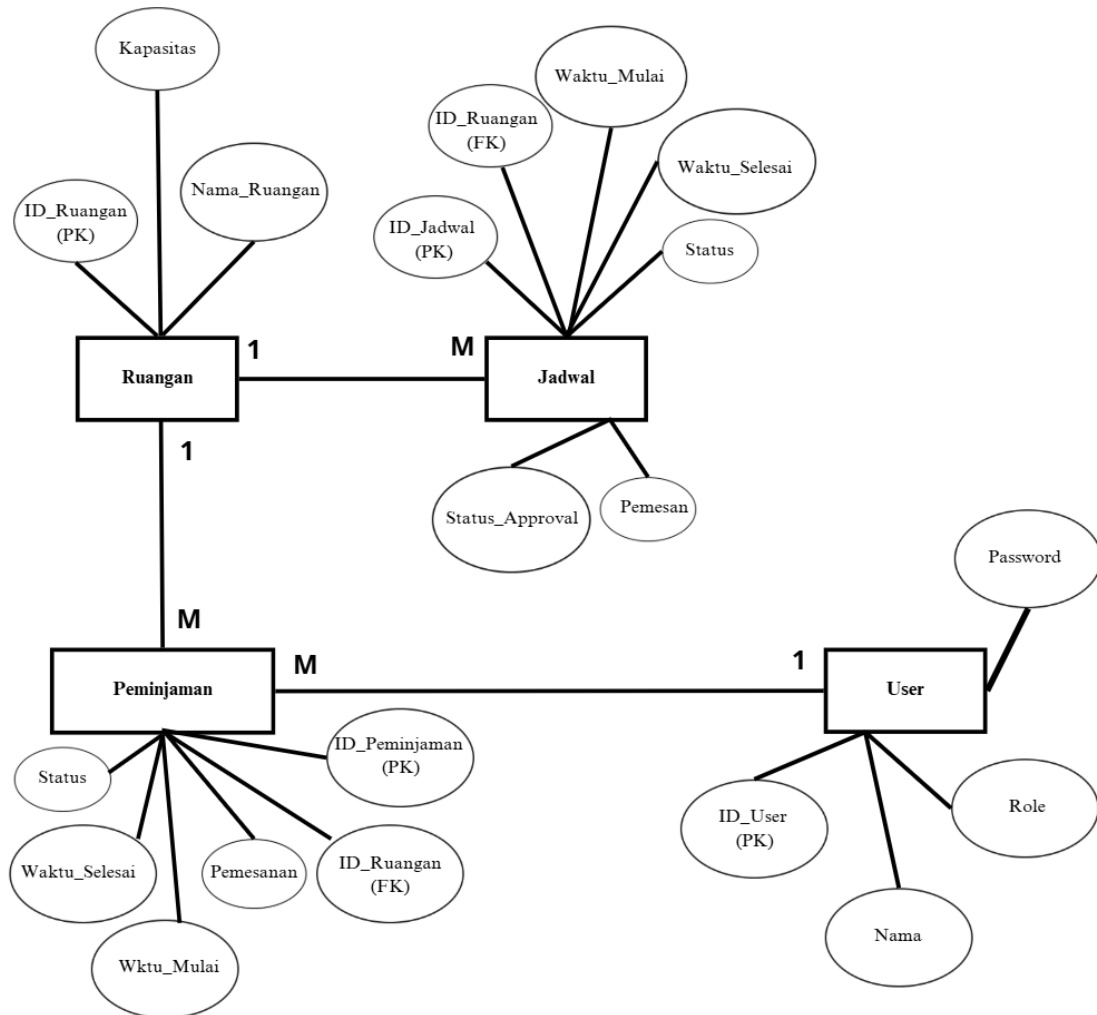
b. Relasi Antar Entitas

1. User ↔ Peminjaman
 - Relasi: Seorang User (Mahasiswa/Dosen) dapat mengajukan banyak Peminjaman.
 - Tipe Relasi: One-to-Many (1:N)
2. Ruangan ↔ Jadwal
 - Relasi: Satu Ruangan dapat memiliki banyak Jadwal.
 - Tipe Relasi: One-to-Many (1:N)
3. Ruangan ↔ Peminjaman
 - Relasi: Satu Ruangan dapat memiliki banyak Peminjaman.
 - Tipe Relasi: One-to-Many (1:N)

Penjelasan Relasi:

1. User ke Peminjaman:
 - Setiap peminjaman dilakukan oleh satu pengguna (Mahasiswa/Dosen).
 - Seorang pengguna dapat memiliki banyak peminjaman.
2. Ruangan ke Jadwal:

- Setiap jadwal terkait dengan satu ruangan.
 - Satu ruangan dapat memiliki banyak jadwal.
3. Ruangan ke Peminjaman:
- Setiap peminjaman terkait dengan satu ruangan.
 - Satu ruangan dapat memiliki banyak peminjaman.



2.7 Program

Berikut merupakan program yang telah kami buat untuk membuat sistem manajemen ruangan kampus berbasis python dan SQL.

```
import sqlite3
import tkinter as tk
from tkinter import messagebox, ttk
import random
import string
from datetime import datetime

# Fungsi untuk menghasilkan ID unik
def generate_id(prefix="ID", length=8):
    characters = string.ascii_uppercase + string.digits
    return prefix + ''.join(random.choices(characters, k=length))

# Kelas untuk menangani operasi database
class DatabaseHandler:
    def __init__(self, db_name="ruangan_kampus.db"):
        self.db_name = db_name
        self.init_db()

    def init_db(self):
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            # Hapus tabel Jadwal jika sudah ada (opsional)
            # cursor.execute("DROP TABLE IF EXISTS Jadwal")

            # Membuat tabel Ruangan
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS Ruangan (
                ID_Ruangan TEXT PRIMARY KEY,
                Nama_Ruangan TEXT NOT NULL,
                Kapasitas INTEGER NOT NULL
            )
            """)

            # Membuat tabel User
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS User (
                ID_User TEXT PRIMARY KEY,
                Nama TEXT NOT NULL,
                Password TEXT NOT NULL,
                Role TEXT CHECK(Role IN ('Operator', 'Mahasiswa',
'Dosen')) NOT NULL
            )
            """)

            # Membuat tabel Jadwal
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS Jadwal (
                ID_Jadwal TEXT PRIMARY KEY,
                ID_Ruangan TEXT NOT NULL,
                Waktu_Mulai TEXT NOT NULL,
                Waktu_Selesai TEXT NOT NULL,
```

```

        Status TEXT CHECK(Status IN ('Tersedia', 'Dipakai',
'Terbooking')) NOT NULL,
        Pemesan TEXT,
        Status_Approval TEXT CHECK(Status_Approval IN
('Approved', 'Pending', 'Rejected')) DEFAULT 'Pending',
        FOREIGN KEY (ID_Ruangan) REFERENCES
Ruangan(ID_Ruangan)
    )
    """)

# Membuat tabel Peminjaman
cursor.execute("""
CREATE TABLE IF NOT EXISTS Peminjaman (
    ID_Peminjaman TEXT PRIMARY KEY,
    ID_Ruangan TEXT NOT NULL,
    Waktu_Mulai TEXT NOT NULL,
    Waktu_Selesai TEXT NOT NULL,
    Pemesan TEXT,
    Status TEXT CHECK(Status IN ('Menunggu', 'Disetujui',
'Ditolak')) DEFAULT 'Menunggu',
    FOREIGN KEY (ID_Ruangan) REFERENCES
Ruangan(ID_Ruangan)
)
""")

conn.commit()

def tambah_user(self, id_user, nama, password, role):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("INSERT INTO User (ID_User, Nama,
Password, Role) VALUES (?, ?, ?, ?)",
                (id_user, nama, password, role))
            conn.commit()
    except sqlite3.IntegrityError:
        # User sudah ada, tidak melakukan apa-apa
        pass
    except sqlite3.OperationalError as e:
        print(f"Error: {e}")

def setup_default_users(self):
    default_users = [
        ("O001", "Operator1", "password", "Operator"),
        ("M001", "Mahasiswa1", "password", "Mahasiswa"),
        ("D001", "Dosen1", "password", "Dosen")
    ]
    for user in default_users:
        self.tambah_user(*user)

def autentikasi_user(self, id_user, password):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM User WHERE ID_User = ?",
(id_user,))

            user = cursor.fetchone()

            if user and user[2] == password:

```

```

        return user[3] # Mengembalikan role
    else:
        return None
except sqlite3.OperationalError as e:
    messagebox.showerror("Error", f"Error database: {e}")
    return None

def get_ruangan(self):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM Ruangan")
            return cursor.fetchall()
    except sqlite3.OperationalError as e:
        messagebox.showerror("Error", f"Error database: {e}")
        return []

def get_daftar_ruangan(self):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT ID_Ruangan, Nama_Ruangan FROM
Ruangan")
            return cursor.fetchall()
    except sqlite3.OperationalError as e:
        messagebox.showerror("Error", f"Error database: {e}")
        return []

def get_jadwal(self):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("""
                SELECT
                    j.ID_Jadwal, -- Menambahkan ID_Jadwal
                    r>Nama_Ruangan,
                    IFNULL(j.Waktu_Mulai, 'Belum Ada Jadwal') AS
Waktu_Mulai,
                    IFNULL(j.Waktu_Selesai, 'Belum Ada Jadwal') AS
Waktu_Selesai,
                    IFNULL(j.Status, 'Tersedia') AS Status,
                    IFNULL(j.Pemesan, 'Belum Ada') AS Pemesan,
                    IFNULL(j.Status_Approval, '-') AS
Status_Approval
                FROM Ruangan r
                LEFT JOIN Jadwal j ON r.ID_Ruangan = j.ID_Ruangan
                ORDER BY r.ID_Ruangan, j.Waktu_Mulai
            """)
            return cursor.fetchall()
    except sqlite3.OperationalError as e:
        messagebox.showerror("Error", f"Operasi database gagal:
{e}")
        return []

def tambah_ruangan_ke_db(self, id_ruangan, nama_ruangan,
kapasitas):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()

```

```

        cursor.execute("""
            INSERT INTO Ruangan (ID_Ruangan, Nama_Ruangan,
Kapasitas)
                VALUES (?, ?, ?)
            """, (id_ruangan, nama_ruangan, kapasitas))
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        messagebox.showerror("Error", "ID Ruangan sudah ada!
Silakan gunakan ID yang berbeda.")
        return False
    except sqlite3.OperationalError as e:
        messagebox.showerror("Error", f"Operasi database gagal:
{e}")
        return False

    def ajukan_peminjaman(self, id_ruangan, waktu_mulai,
waktu_selesai, pemesanan):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                id_peminjaman = generate_id(prefix="P")
                cursor.execute("""
                    INSERT INTO Peminjaman (ID_Peminjaman, ID_Ruangan,
Waktu_Mulai, Waktu_Selesai, Pemesan, Status)
                        VALUES (?, ?, ?, ?, ?, ?)
                    """, (id_peminjaman, id_ruangan, waktu_mulai,
waktu_selesai, pemesanan, 'Menunggu'))
                conn.commit()
                return True
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal:
{e}")
            return False

    def approve_reject_peminjaman(self, id_peminjaman, status):
        if status not in ["Disetujui", "Ditolak"]:
            messagebox.showerror("Error", "Status tidak valid!")
            return False
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                # Jika ingin menyetujui, periksa konflik jadwal
                if status == "Disetujui":
                    cursor.execute("""
                        SELECT ID_Ruangan, Waktu_Mulai, Waktu_Selesai
                        FROM Peminjaman
                        WHERE ID_Peminjaman = ?
                    """, (id_peminjaman,))
                    peminjaman = cursor.fetchone()
                    if peminjaman:
                        id_ruangan, waktu_mulai, waktu_selesai =
peminjaman
                        if self.is_jadwal_conflict(id_ruangan,
waktu_mulai, waktu_selesai):
                            messagebox.showerror("Konflik Jadwal",
"Waktu peminjaman bertabrakan dengan jadwal yang sudah ada.")
                            return False

```



```

        # Update status di tabel Peminjaman
        cursor.execute("""
            UPDATE Peminjaman
            SET Status = ?
            WHERE ID_Peminjaman = ?
            """, (status, id_peminjaman))

        if status == "Disetujui":
            # Ambil detail peminjaman yang disetujui
            cursor.execute("""
                SELECT ID_Ruangan, Waktu_Mulai, Waktu_Selesai,
Pemesan
                FROM Peminjaman
                WHERE ID_Peminjaman = ?
            """, (id_peminjaman,))
            peminjaman = cursor.fetchone()

            if peminjaman:
                id_ruangan, waktu_mulai, waktu_selesai,
pemesan = peminjaman

                # Buat entry baru di tabel Jadwal
                id_jadwal = generate_id(prefix="J")
                cursor.execute("""
                    INSERT INTO Jadwal (ID_Jadwal, ID_Ruangan,
Waktu_Mulai, Waktu_Selesai, Status, Pemesan, Status_Approval)
                    VALUES (?, ?, ?, ?, ?, ?, 'Approved')
                """, (id_jadwal, id_ruangan, waktu_mulai,
waktu_selesai, 'Terbooking', pemesan))
                print(f"Jadwal berhasil ditambahkan dengan
ID_Jadwal: {id_jadwal}") # Debug
                messagebox.showinfo("Sukses", f"Jadwal
berhasil ditambahkan dengan ID_Jadwal: {id_jadwal}")
                conn.commit()
                return True
            except sqlite3.OperationalError as e:
                messagebox.showerror("Error", f"Operasi database gagal:
{e}")
                return False

        def is_jadwal_conflict(self, id_ruangan, waktu_mulai,
waktu_selesai):
            try:
                with sqlite3.connect(self.db_name, timeout=10) as conn:
                    cursor = conn.cursor()
                    cursor.execute("""
                        SELECT *
                        FROM Jadwal
                        WHERE ID_Ruangan = ?
                        AND Status = 'Terbooking'
                        AND (
                            (? < Waktu_Selesai AND ? > Waktu_Mulai)
                        )
                    """, (id_ruangan, waktu_mulai, waktu_selesai))
                    return cursor.fetchone() is not None
            except sqlite3.OperationalError as e:
                messagebox.showerror("Error", f"Operasi database gagal:
{e}")
                return True # Asumsi terjadi konflik jika gagal mengecek

```

```

def get_pengajuan_peminjaman(self):
    try:
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            cursor.execute("""
                SELECT p.ID_Peminjaman, r.Nama_Ruangan,
p.Waktu_Mulai, p.Waktu_Selesai, p.Pemesan, p.Status
                FROM Peminjaman p
                JOIN Ruangan r ON p.ID_Ruangan = r.ID_Ruangan
            """)
            return cursor.fetchall()
    except sqlite3.OperationalError as e:
        messagebox.showerror("Error", f"Error database: {e}")
        return []

# Kelas untuk menangani autentikasi
class Auth:
    def __init__(self, db_handler):
        self.db_handler = db_handler

    def authenticate(self, id_user, password):
        return self.db_handler.autentikasi_user(id_user, password)

# Kelas UI Login
class LoginUI:
    def __init__(self, root, auth_handler, on_success):
        self.root = root
        self.auth_handler = auth_handler
        self.on_success = on_success
        self.setup_ui()

    def setup_ui(self):
        self.root.title("Login")

        tk.Label(self.root, text="ID User:").grid(row=0, column=0,
padx=10, pady=10)
        self.entry_id_user = tk.Entry(self.root)
        self.entry_id_user.grid(row=0, column=1, padx=10, pady=10)

        tk.Label(self.root, text="Password:").grid(row=1, column=0,
padx=10, pady=10)
        self.entry_password = tk.Entry(self.root, show="*")
        self.entry_password.grid(row=1, column=1, padx=10, pady=10)

        tk.Button(self.root, text="Login",
command=self.cek_login).grid(row=2, column=0, columnspan=2, pady=10)

    def cek_login(self):
        id_user = self.entry_id_user.get()
        password = self.entry_password.get()
        role = self.auth_handler.authenticate(id_user, password)

        if role:
            self.on_success(role)
            # Clear the entries
            self.entry_id_user.delete(0, tk.END)
            self.entry_password.delete(0, tk.END)
        else:

```

```

        messagebox.showerror("Login Gagal", "ID atau Password
salah!")

# Kelas UI Operator
class OperatorUI:
    def __init__(self, db_handler, master):
        self.db_handler = db_handler
        self.master = master
        self.window = tk.Toplevel(master)
        self.window.title("Operator - Manajemen Ruangan dan Jadwal")
        self.setup_ui()

    def setup_ui(self):
        # Frame untuk Ruangan
        frame_ruangan = tk.LabelFrame(self.window, text="Data
Ruangan")
        frame_ruangan.pack(fill="both", expand=True, padx=10, pady=5)

        columns_ruangan = ("ID Ruangan", "Nama Ruangan", "Kapasitas")
        self.tree_ruangan = ttk.Treeview(frame_ruangan,
columns=columns_ruangan, show="headings")
        for col in columns_ruangan:
            self.tree_ruangan.heading(col, text=col)
        self.tree_ruangan.pack(fill="both", expand=True)

        tk.Button(self.window, text="Tambah Ruangan",
command=self.tambah_ruangan_ui).pack(pady=5)

        # Frame untuk Jadwal
        frame_jadwal = tk.LabelFrame(self.window, text="Data Jadwal")
        frame_jadwal.pack(fill="both", expand=True, padx=10, pady=5)

        columns_jadwal = ("ID Jadwal", "Nama Ruangan", "Waktu Mulai",
"Waktu Selesai", "Status", "Pemesan", "Status Approval")
        self.tree_jadwal = ttk.Treeview(frame_jadwal,
columns=columns_jadwal, show="headings")
        for col in columns_jadwal:
            self.tree_jadwal.heading(col, text=col)
        self.tree_jadwal.pack(fill="both", expand=True)

        tk.Button(self.window, text="Tampilkan Jadwal",
command=self.tampilkan_data_jadwal).pack(pady=5)

        # Frame untuk Pengajuan Peminjaman
        frame_pengajuan = tk.LabelFrame(self.window, text="Pengajuan
Peminjaman")
        frame_pengajuan.pack(fill="both", expand=True, padx=10,
pady=5)

        columns_pengajuan = ("ID Peminjaman", "Nama Ruangan", "Waktu
Mulai", "Waktu Selesai", "Pemesan", "Status")
        self.tree_pengajuan = ttk.Treeview(frame_pengajuan,
columns=columns_pengajuan, show="headings")
        for col in columns_pengajuan:
            self.tree_pengajuan.heading(col, text=col)
        self.tree_pengajuan.pack(fill="both", expand=True)

        tk.Button(self.window, text="Tampilkan Pengajuan",
command=self.tampilkan_pengajuan_peminjaman).pack(pady=5)

```

```

        tk.Button(self.window, text="Approve/Reject",
command=self.approve_reject_ui).pack(pady=5)

        # Tombol Refresh Data
        tk.Button(self.window, text="Refresh Data",
command=self.refresh_all).pack(pady=10)

        # Menampilkan data awal
        self.tampilkan_data_ruangan()
        self.tampilkan_data_jadwal()
        self.tampilkan_pengajuan_peminjaman()

def refresh_all(self):
    self.tampilkan_data_ruangan()
    self.tampilkan_data_jadwal()
    self.tampilkan_pengajuan_peminjaman()

def tampilkan_data_ruangan(self):
    for item in self.tree_ruangan.get_children():
        self.tree_ruangan.delete(item)
    data = self.db_handler.get_ruangan()
    for row in data:
        self.tree_ruangan.insert("", tk.END, values=row)

def tampilkan_data_jadwal(self):
    for item in self.tree_jadwal.get_children():
        self.tree_jadwal.delete(item)
    data = self.db_handler.get_jadwal()
    for row in data:
        self.tree_jadwal.insert("", tk.END, values=row)

def tampilkan_pengajuan_peminjaman(self):
    for item in self.tree_pengajuan.get_children():
        self.tree_pengajuan.delete(item)
    data = self.db_handler.get_pengajuan_peminjaman()
    for row in data:
        self.tree_pengajuan.insert("", tk.END, values=row)

def approve_reject_ui(self):
    selected_item = self.tree_pengajuan.selection()
    if not selected_item:
        messagebox.showwarning("Pilih Pengajuan", "Pilih pengajuan
peminjaman yang akan diproses!")
        return

    id_peminjaman = self.tree_pengajuan.item(selected_item,
"values")[0]

    def approve():
        if
self.db_handler.approve_reject_peminjaman(id_peminjaman, "Disetujui"):
            messagebox.showinfo("Sukses", "Peminjaman berhasil
disetujui!")

            self.tampilkan_pengajuan_peminjaman()
            self.tampilkan_data_jadwal() # Refresh jadwal
            form.destroy()

    def reject():

```

```

        if
self.db_handler.approve_reject_peminjaman(id_peminjaman, "Ditolak"):
        messagebox.showinfo("Sukses", "Peminjaman berhasil
ditolak!")

        self.tampilkan_pengajuan_peminjaman()
        form.destroy()

        form = tk.Toplevel(self.window)
        form.title("Konfirmasi Approve/Reject")
        tk.Label(form, text="Apakah Anda ingin menyetujui atau menolak
peminjaman ini?").pack(pady=10)
        tk.Button(form, text="Approve", command=approve).pack(pady=5)
        tk.Button(form, text="Reject", command=reject).pack(pady=5)

    def tambah_ruangan_ui(self):
    def simpan_ruangan():
        id_ruangan = entry_id_ruangan.get().strip()
        nama_ruangan = entry_nama_ruangan.get().strip()
        kapasitas = entry_kapasitas.get().strip()

        if not id_ruangan or not nama_ruangan or not kapasitas:
            messagebox.showwarning("Input Kosong", "Semua kolom
harus diisi!")
            return

        try:
            kapasitas = int(kapasitas)
            if kapasitas <= 0:
                raise ValueError
        except ValueError:
            messagebox.showwarning("Input Salah", "Kapasitas harus
berupa angka positif!")
            return

        if self.db_handler.tambah_ruangan_ke_db(id_ruangan,
nama_ruangan, kapasitas):
            messagebox.showinfo("Sukses", "Ruangan berhasil
ditambahkan!")

            form.destroy()
            self.tampilkan_data_ruangan()

        form = tk.Toplevel(self.window)
        form.title("Tambah Ruangan")

        tk.Label(form, text="ID Ruangan:").grid(row=0, column=0,
padx=10, pady=5)
        entry_id_ruangan = tk.Entry(form)
        entry_id_ruangan.grid(row=0, column=1, padx=10, pady=5)

        tk.Label(form, text="Nama Ruangan:").grid(row=1, column=0,
padx=10, pady=5)
        entry_nama_ruangan = tk.Entry(form)
        entry_nama_ruangan.grid(row=1, column=1, padx=10, pady=5)

        tk.Label(form, text="Kapasitas:").grid(row=2, column=0,
padx=10, pady=5)
        entry_kapasitas = tk.Entry(form)
        entry_kapasitas.grid(row=2, column=1, padx=10, pady=5)

```

```

        tk.Button(form, text="Simpan Ruangan",
command=simpan_ruangan).grid(row=3, column=0, columnspan=2, pady=10)

# Kelas UI Mahasiswa dan Dosen
class MahasiswaDosenUI:
    def __init__(self, db_handler, role, master):
        self.db_handler = db_handler
        self.role = role
        self.master = master
        self.window = tk.Toplevel(master)
        self.window.title(f"Manajemen Jadwal - {self.role}")
        self.setup_ui()

    def setup_ui(self):
        # Frame untuk Daftar Ruangan
        frame_daftar_ruangan = tk.LabelFrame(self.window, text="Daftar
Ruangan")
        frame_daftar_ruangan.pack(fill="both", expand=True, padx=10,
pady=5)

        columns_daftar_ruangan = ("ID Ruangan", "Nama Ruangan")
        self.tree_daftar_ruangan = ttk.Treeview(frame_daftar_ruangan,
columns=columns_daftar_ruangan, show="headings")
        for col in columns_daftar_ruangan:
            self.tree_daftar_ruangan.heading(col, text=col)
        self.tree_daftar_ruangan.pack(fill="both", expand=True)

        # Tombol Refresh Daftar Ruangan
        tk.Button(self.window, text="Refresh Daftar Ruangan",
command=self.tampilkan_daftar_ruangan).pack(pady=5)

        # Frame untuk Jadwal
        frame_jadwal = tk.LabelFrame(self.window, text="Data Jadwal")
        frame_jadwal.pack(fill="both", expand=True, padx=10, pady=5)

        columns_jadwal = ("ID Jadwal", "Nama Ruangan", "Waktu Mulai",
"Waktu Selesai", "Status", "Pemesan", "Status Approval")
        self.tree_jadwal = ttk.Treeview(frame_jadwal,
columns=columns_jadwal, show="headings")
        for col in columns_jadwal:
            self.tree_jadwal.heading(col, text=col)
        self.tree_jadwal.pack(fill="both", expand=True)

        # Tombol untuk Lihat Jadwal
        tk.Button(self.window, text="Lihat Jadwal",
command=self.tampilkan_data_jadwal).pack(pady=5)
        tk.Button(self.window, text="Ajukan Peminjaman",
command=self.ajukan_peminjaman_ui).pack(pady=5)
        tk.Button(self.window, text="Refresh Data",
command=self.tampilkan_data_jadwal).pack(pady=5)

        # Menampilkan data awal
        self.tampilkan_daftar_ruangan()
        self.tampilkan_data_jadwal()

    def tampilkan_daftar_ruangan(self):
        for item in self.tree_daftar_ruangan.get_children():
            self.tree_daftar_ruangan.delete(item)
        data = self.db_handler.get_daftar_ruangan()

```

```

        for row in data:
            self.tree_daftar_ruangan.insert("", tk.END, values=row)

    def tampilkan_data_jadwal(self):
        for item in self.tree_jadwal.get_children():
            self.tree_jadwal.delete(item)
        data = self.db_handler.get_jadwal()
        for row in data:
            self.tree_jadwal.insert("", tk.END, values=row)

    def ajukan_peminjaman_ui(self):
        def simpan_peminjaman():
            # Mengambil ID Ruangan dari Combobox
            selected_ruangan = combo_ruangan.get()
            if not selected_ruangan:
                messagebox.showwarning("Input Kosong", "Silakan pilih
ruangan!")
            return
            id_ruangan = selected_ruangan.split(" - ")[0] # Asumsi
format "ID_Ruangan - Nama_Ruangan"

            waktu_mulai = entry_waktu_mulai.get().strip()
            waktu_selesai = entry_waktu_selesai.get().strip()
            pemesanan = entry_pemesan.get().strip()

            if not waktu_mulai or not waktu_selesai or not pemesanan:
                messagebox.showwarning("Input Kosong", "Semua kolom
harus diisi!")
            return

            # Validasi format waktu (misalnya, YYYY-MM-DD HH:MM)
            try:
                datetime.strptime(waktu_mulai, "%Y-%m-%d %H:%M")
                datetime.strptime(waktu_selesai, "%Y-%m-%d %H:%M")
            except ValueError:
                messagebox.showwarning("Format Salah", "Format waktu
harus YYYY-MM-DD HH:MM")
            return

            if self.db_handler.ajukan_peminjaman(id_ruangan,
waktu_mulai, waktu_selesai, pemesanan):
                messagebox.showinfo("Sukses", "Peminjaman berhasil
diajukan!")
                form.destroy()
                self.tampilkan_data_jadwal()

        form = tk.Toplevel(self.window)
        form.title("Ajukan Peminjaman")

        tk.Label(form, text="Ruangan:").grid(row=0, column=0, padx=10,
pady=5)

        # Mengambil daftar ruangan untuk mengisi Combobox
        daftar_ruangan = self.db_handler.get_daftar_ruangan()
        if not daftar_ruangan:
            messagebox.showerror("Error", "Tidak ada ruangan yang
tersedia!")
            form.destroy()
            return

```

```

        ruangan_options = [f"{ruangan[0]} - {ruangan[1]}" for ruangan
in daftar_ruangan]
        combo_ruangan = ttk.Combobox(form, values=ruangan_options,
state="readonly")
        combo_ruangan.grid(row=0, column=1, padx=10, pady=5)
        combo_ruangan.current(0) # Pilih ruangan pertama secara
default

        tk.Label(form, text="Waktu Mulai (YYYY-MM-DD
HH:MM):").grid(row=1, column=0, padx=10, pady=5)
        entry_waktu_mulai = tk.Entry(form)
        entry_waktu_mulai.grid(row=1, column=1, padx=10, pady=5)

        tk.Label(form, text="Waktu Selesai (YYYY-MM-DD
HH:MM):").grid(row=2, column=0, padx=10, pady=5)
        entry_waktu_selesai = tk.Entry(form)
        entry_waktu_selesai.grid(row=2, column=1, padx=10, pady=5)

        tk.Label(form, text="Pemesan:").grid(row=3, column=0, padx=10,
pady=5)
        entry_pemesan = tk.Entry(form)
        entry_pemesan.grid(row=3, column=1, padx=10, pady=5)

        tk.Button(form, text="Ajukan Peminjaman",
command=simpan_peminjaman).grid(row=4, column=0, columnspan=2,
pady=10)

# Kelas untuk menjalankan aplikasi
class RoomManagementApp:
    def __init__(self):
        self.db_handler = DatabaseHandler()
        self.db_handler.setup_default_users()
        self.auth = Auth(self.db_handler)
        self.root = tk.Tk()
        self.root.geometry("300x150")
        self.login_ui = LoginUI(self.root, self.auth,
self.on_login_success)
        self.root.mainloop()

    def on_login_success(self, role):
        if role == "Operator":
            OperatorUI(self.db_handler, self.root)
        elif role in ["Mahasiswa", "Dosen"]:
            MahasiswaDosenUI(self.db_handler, role, self.root)
        else:
            messagebox.showerror("Error", "Role tidak dikenali!")

# Menjalankan aplikasi
if __name__ == "__main__":
    RoomManagementApp()

```

Tampilan dari hasil program terlampir
Berikut merupakan SQL dari program kami:

-- Hapus tabel jika sudah ada sebelumnya (opsional)


```

DROP TABLE IF EXISTS Ruangan;
DROP TABLE IF EXISTS User;
DROP TABLE IF EXISTS Jadwal;
DROP TABLE IF EXISTS Peminjaman;

-- Membuat tabel Ruangan
CREATE TABLE IF NOT EXISTS Ruangan (
    ID_Ruangan TEXT PRIMARY KEY,
    Nama_Ruangan TEXT NOT NULL,
    Kapasitas INTEGER NOT NULL
);

-- Membuat tabel User
CREATE TABLE IF NOT EXISTS User (
    ID_User TEXT PRIMARY KEY,
    Nama TEXT NOT NULL,
    Password TEXT NOT NULL,
    Role TEXT CHECK(Role IN ('Operator', 'Mahasiswa', 'Dosen')) NOT
NULL
);

-- Membuat tabel Jadwal
CREATE TABLE IF NOT EXISTS Jadwal (
    ID_Jadwal TEXT PRIMARY KEY,
    ID_Ruangan TEXT NOT NULL,
    Waktu_Mulai TEXT NOT NULL,
    Waktu_Selesai TEXT NOT NULL,
    Status TEXT CHECK(Status IN ('Tersedia', 'Dipakai', 'Terbooking'))
NOT NULL,
    Pemesan TEXT,
    Status_Approval TEXT CHECK(Status_Approval IN ('Approved',
'Pending', 'Rejected')) DEFAULT 'Pending',
    FOREIGN KEY (ID_Ruangan) REFERENCES Ruangan(ID_Ruangan)
);

-- Membuat tabel Peminjaman
CREATE TABLE IF NOT EXISTS Peminjaman (
    ID_Peminjaman TEXT PRIMARY KEY,
    ID_Ruangan TEXT NOT NULL,
    Waktu_Mulai TEXT NOT NULL,
    Waktu_Selesai TEXT NOT NULL,
    Pemesan TEXT,
    Status TEXT CHECK(Status IN ('Menunggu', 'Disetujui', 'Ditolak'))
DEFAULT 'Menunggu',
    FOREIGN KEY (ID_Ruangan) REFERENCES Ruangan(ID_Ruangan)
);

-- Menambahkan data awal ke tabel User
INSERT INTO User (ID_User, Nama, Password, Role) VALUES
('O001', 'Operator1', 'password', 'Operator'),
('M001', 'Mahasiswa1', 'password', 'Mahasiswa'),
('D001', 'Dosen1', 'password', 'Dosen');

-- Contoh query untuk mengambil data dari tabel Ruangan
-- SELECT * FROM Ruangan;

-- Contoh query untuk mengambil data Jadwal dengan join ke Ruangan
-- SELECT
--     j.ID_Jadwal,

```

```
--      r>Nama_Ruangan,
--      IFNULL(j.Waktu_Mulai, 'Belum Ada Jadwal') AS Waktu_Mulai,
--      IFNULL(j.Waktu_Selesai, 'Belum Ada Jadwal') AS Waktu_Selesai,
--      IFNULL(j.Status, 'Tersedia') AS Status,
--      IFNULL(j.Pemesan, 'Belum Ada') AS Pemesan,
--      IFNULL(j.Status_Approval, '-') AS Status_Approval
-- FROM Ruangan r
-- LEFT JOIN Jadwal j ON r.ID_Ruangan = j.ID_Ruangan
-- ORDER BY r.ID_Ruangan, j.Waktu_Mulai;

-- Contoh query untuk menambahkan ruangan ke tabel
-- INSERT INTO Ruangan (ID_Ruangan, Nama_Ruangan, Kapasitas) VALUES
('R001', 'Ruang Kelas A', 50);

-- Contoh query untuk mengajukan peminjaman
-- INSERT INTO Peminjaman (ID_Peminjaman, ID_Ruangan, Waktu_Mulai,
Waktu_Selesai, Pemesan, Status)
-- VALUES ('P001', 'R001', '2024-12-10 08:00', '2024-12-10 10:00',
'Mahasiswa', 'Menunggu');
```

2.8 Query SQL

Berikut merupakan Query SQL dan tampilan hasil dari Query terlampir pada lampiran

2.8.1 Laporan penggunaan ruangan per hari

(Menampilkan jumlah peminjaman yang di setujui untuk setiap ruangan pada hari tertentu)

```
SELECT
    r>Nama_Ruangan,
    COUNT(p.ID_Peminjaman) AS Jumlah_Peminjaman,
    DATE(p.Waktu_Mulai) AS Tanggal
FROM
    Ruangan r
LEFT JOIN
    Peminjaman p ON r.ID_Ruangan = p.ID_Ruangan
WHERE
    p.Status = 'Disetujui'
    AND DATE(p.Waktu_Mulai) = '2024-12-15'
GROUP BY
    r>Nama_Ruangan, DATE(p.Waktu_Mulai)
ORDER BY
    r>Nama_Ruangan;
```

Penjelasan:

- Menghitung jumlah peminjaman yang disetujui per ruangan pada tanggal 15 Desember 2024.
- Menggunakan `LEFT JOIN` untuk memastikan semua ruangan ditampilkan meskipun tidak ada peminjaman.

2.8.2 Laporan Peminjaman yang Menunggu Persetujuan

(Menampilkan semua pengajuan peminjaman yang masih dalam status "Menunggu")

```
SELECT
```

```

        p.ID_Peminjaman,
        r>Nama_Ruangan,
        p.Waktu_Mulai,
        p.Waktu_Selesai,
        p.Pemesan
FROM
    Peminjaman p
JOIN
    Ruangan r ON p.ID_Ruangan = r.ID_Ruangan
WHERE
    p.Status = 'Menunggu'
ORDER BY
    p.Waktu_Mulai ASC;

```

Penjelasan:

- Menampilkan semua pengajuan peminjaman yang statusnya masih "Menunggu" untuk ditindaklanjuti oleh operator.
- Mengurutkan berdasarkan waktu mulai peminjaman.

2.8.3 Laporan Rekapitulasi Peminjaman Mingguan

(Menampilkan total peminjaman per ruangan selama seminggu)

```

SELECT
    r>Nama_Ruangan,
    COUNT(p.ID_Peminjaman) AS Total_Peminjaman,
    STRFTIME('%W', p.Waktu_Mulai) AS Minggu_Tahun
FROM
    Ruangan r
JOIN
    Peminjaman p ON r.ID_Ruangan = p.ID_Ruangan
WHERE
    p.Status = 'Disetujui'
    AND p.Waktu_Mulai BETWEEN DATE('2024-12-09') AND DATE('2024-12-15')
GROUP BY
    r>Nama_Ruangan, STRFTIME('%W', p.Waktu_Mulai)
ORDER BY
    r>Nama_Ruangan, Minggu_Tahun;

```

Penjelasan:

- Menghitung total peminjaman yang disetujui per ruangan selama minggu ke-50 tahun 2024 (asumsi tanggal 09-12-2024 sampai 15-12-2024).
- Menggunakan fungsi STRFTIME untuk mendapatkan nomor minggu dari tanggal peminjaman.

BAB III

PENUTUP

3.1 Kesimpulan

- Sistem manajemen ruangan kampus berbasis Python dan SQL yang dikembangkan mampu mencatat, mengelola, dan melaporkan data penggunaan ruangan secara otomatis. Hal ini berpotensi mengurangi konflik jadwal dan meningkatkan efisiensi operasional.
- Sistem dilengkapi autentikasi pengguna, sehingga data terjaga dengan aman dan aktivitas pengguna tercatat dengan jelas untuk memudahkan audit.
- Fungsi ekspor ke format CSV memberikan kemudahan dalam analisis lebih lanjut, baik untuk laporan harian, mingguan, maupun bulanan.
- Proyek ini berhasil menerapkan konsep ERD dan normalisasi, memberikan pengalaman langsung kepada mahasiswa dalam perancangan sistem berbasis data.

3.2 Saran

- Untuk meningkatkan pengalaman pengguna, sistem dapat dikembangkan dari antarmuka berbasis teks (CLI) menjadi berbasis GUI (Graphical User Interface).
- Sistem ini dapat diintegrasikan dengan platform manajemen kampus lainnya, seperti portal akademik atau sistem keuangan, untuk penggunaan yang lebih menyeluruh.
- Uji coba pada sistem dapat diperluas mencakup simulasi beban tinggi untuk memastikan kehandalan ketika digunakan oleh banyak pengguna secara bersamaan.
- Menambahkan fitur notifikasi (email atau pesan) untuk memberitahukan status peminjaman atau pengingat jadwal kepada pengguna.
- Menyediakan dokumentasi yang lebih lengkap untuk pengembang dan panduan yang mudah dipahami bagi pengguna akhir.

3.3 Pembagian Tugas

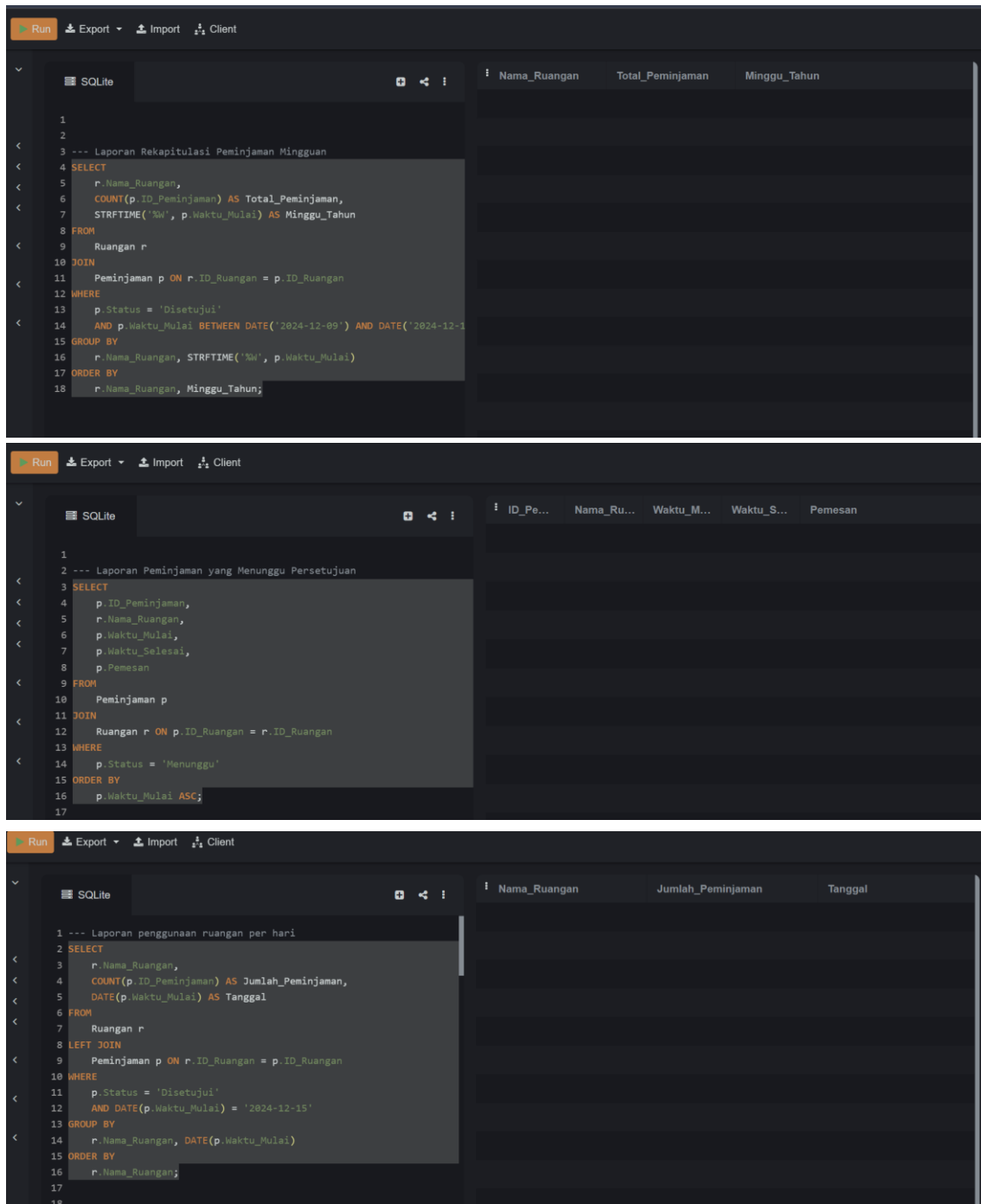
- | | |
|-------------------------------------|--------------------------------|
| 1. Anisah Ainulhani (13423122) | : PPT |
| 2. Najmi Rifdah Hanifah (13423123) | : Penyusunan laporan dan video |
| 3. M. Yasin Nurfadllilah (13423136) | : Pembuatan program |

DAFTAR PUSTAKA

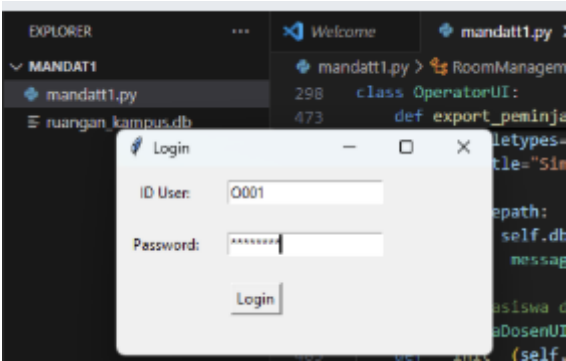
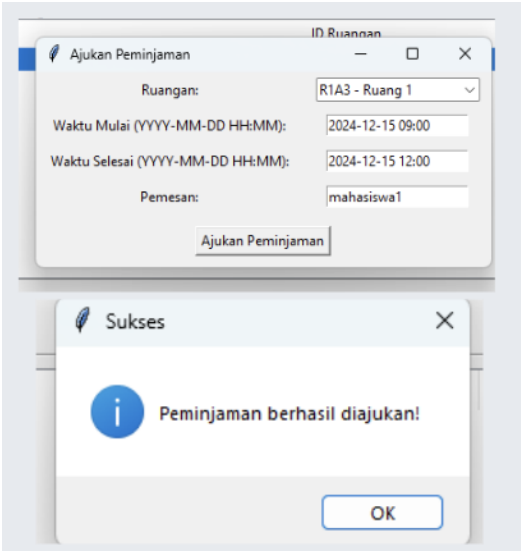
Andika, D. (2017). Pengertian Flowchart. Diakses pada November 29, 2024, from <https://www.it-jurnal.com/pengertian-flowchart/#:~:text=Flowchart%20Adalah%20Bagan%2Dbagan%20yang,cara%20penyajian%20dari%20suatu%20algoritma.>

Ismail, I. (2021, Oktober 14). accurate. Retrieved from Entity Relationship Diagram: Pengertian, Fungsi, dan Cara Membuatnya: <https://accurate.id/marketing-manajemen/entity-relationship-diagram/>

LAMPIRAN



Berikut beberapa takmpilan dari hasil program



Tampilkan Jadwal							
Pengajuan Peminjaman							
ID Peminjaman	Nama Ruangan	Waktu Mulai	Waktu Selesai	Pemesan	Status		
PETKRSG4	Ruang 1	2024-12-10 08:00	2024-12-10 10:00	1	Menunggu		
POGUXS4CE	Ruang 1	2024-12-15 09:00	2024-12-15 12:00	mahasiswa1	Menunggu		